Pointer-First Agent Constitution and Operational Certification of the Stela Repository

1. Executive Summary: The Epistemology of a Pointer-First Architecture

The Stela repository represents a significant departure from traditional, heuristic-based autonomous software development environments. It establishes a "Pointer-First" constitution—a governance model where authority, logic, and operational constraints are strictly anchored to persistent, addressable files on the filesystem rather than transient prompts or embedded assumptions.[1] This architecture serves to eliminate "ghost canon," the accumulation of unwritten rules that degrade system stability over time. By enforcing a "Flat Truth" topology, Stela ensures that every operational directive is verifiable, traceable, and legally binding within the system's jurisdiction.

This report provides an exhaustive, expert-level certification of the Stela platform as of HEAD: d760e443. The analysis adheres to a strict read-only protocol, auditing the repository's "State of Play" against the "Policy of Truth" (PoT.md) and the active "Task" dashboard (TASK.md). The objective is to verify the structural integrity of the code sources, certify the operational toolchain against available specifications, and identify critical governance gaps where the current state diverges from the prompt's requirement for a fully registered and explainable operational environment.

The investigation confirms a functional "Operational Kernel" (ops/bin/) and a rigorous "Immune System" (tools/lint/). However, the certification identifies a specific "Governance Gap" in the *registration* of the toolchain. While the user-facing binaries are properly specified, the entire executable layer lacks the bureaucratic registration required for "No Ghost" governance. To achieve full certification, the system requires the creation of five specific registries—BINARIES.md, LINT.md, TOOLS.md, TEST.md, and SCRIPTS.md—to bring the toolchain under the same high-fidelity governance as the agent library.

2. Constitutional Architecture and Governance Topology

To certify the individual components of the Stela repository, one must first dissect the constitutional framework that grants them legitimacy. The system operates under a tripartite governance structure defined by the Policy of Truth, the State of Play, and the Task Dashboard. This triad ensures that no agent operates in a vacuum; every action is bounded by

law, recorded in history, and directed by a contract.

2.1 The Policy of Truth (PoT): The Constitutional Core

The PoT.md file functions as the Single Source of Truth (SSOT) for the repository's physical laws.[1] It is not merely a guideline but a rigid constraint mechanism that defines the "Filing Doctrine," segregating the repository into four distinct functional domains:

- **Run (ops/):** The domain of executable logic, manifests, and automation. This is the "kernel" of the repository.
- **Explain (docs/):** The domain of manuals, rationale, and specifications. This provides the semantic backing for the executable logic.
- **Work (projects/):** The domain of payload code. This is where value is generated, subject to the laws of the platform.
- **Trash (storage/):** The domain of ephemeral artifacts and local scratch space, explicitly excluded from canon.

The analysis confirms that PoT.md is actively enforced rather than passively read. The existence of tools/lint/truth.sh demonstrates that the repository possesses a mechanism to police its own governance surfaces, preventing "Canon Drift".[1] The "Pointer-First" mandate is codified here, explicitly prohibiting the duplication of canon text. Instead, agents are required to reference authoritative files (pointers). This is a critical architectural decision: by banning duplication, the system eliminates the risk of conflicting truths. If a rule exists in PoT.md, it must not be restated in TASK.md; it must be pointed to. This reduces the cognitive load on the agent and ensures that updates to the constitution propagate instantly across the system.

2.2 The State of Play (SoP): The Historical Ledger

The SoP.md file serves as the immutable history of the system's evolution.[1] Unlike a simple changelog, the SoP functions as a cryptographic chain of custody for decision-making. It records not just *what* changed, but *why* it changed and *how* it was verified.

Analyzing the recent entries reveals a clear trajectory toward system hardening:

- **DP-OPS-0056:** "Task Subsystem Hardening and Prune Safety Invariants." This entry reflects a focus on preventing the accidental destruction of the active workspace.
- **DP-OPS-0055:** "TASK Surface Contract Lock." This marks the maturation of the task assignment protocol, moving it from experimental to canonical status.
- **DP-OPS-0054:** "TASK Surface Stabilization." This entry records the regression of the open binary to a read-only state regarding tracked files, a critical safety mechanism to prevent the "Freshness Gate" from inadvertently modifying the contract it is meant to verify.

The SoP ensures that the current state of the repository is explainable. The certification process relies on this ledger to verify that tools found in ops/bin are not vestigial artifacts but

active components of recent Dispatch Packets (DPs). For instance, the presence of ops/bin/prune is directly justified by DP-OPS-0047 ("Harden TASK Closeout Cycle and Prune Logic") and further refined in DP-OPS-0051.[1]

2.3 The Task Dashboard (TASK): The Active Interface

TASK.md represents the volatile memory of the system.[1] It acts as the bridge between the operator's intent and the agent's execution. The transition to a "Pointer-First" schema is most evident in the evolution of this file. Recent refactors recorded in the SoP have stripped TASK.md of inline instructions, replacing them with pointers to docs/MANUAL.md and docs/ops/specs/surfaces/task.md.

This architectural shift prevents the "Context Hazard" where an agent follows outdated instructions embedded in a stale task file rather than the canonical manual. The TASK.md file defines the "Freshness Gate," a mandatory check that ensures the agent is operating on the correct branch and HEAD hash before any work begins. This synchronization mechanism is vital for maintaining the integrity of the "Immutable Trunk" doctrine defined in PoT.md.

3. Operational Kernel Certification (ops/bin)

The ops/bin directory constitutes the "Ops Kernel," the executable layer responsible for enforcing the Policy of Truth. Since these binaries are the **Operator Interface**, they must have explicit specifications in docs/ops/specs/binaries/ to explain their behavior.

3.1 ops/bin/context: The Context Archivist

- **Function:** Atomic assembly of session contexts. Bridges the gap between volatile session state (open) and persistent repository state (dump).[1]
- **Specification:** docs/ops/specs/binaries/context.md (Exists).
- **Certification Status: Functional & Specified.** The tool operates within the bounds of PoT.md and fulfills its role in "Context Lifecycle Hardening" (DP-OPS-0035).
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

3.2 ops/bin/dump: The Observer

- **Function:** Creates a deterministic serialization of file system content for LLM ingestion.[1]
- **Specification:** docs/ops/specs/binaries/dump.md (Exists).
- **Certification Status: Critical/Specified.** Foundational to the "Explainability" mandate.
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

3.3 ops/bin/help: The Explanation System

- **Function:** Context-aware search mechanism for documentation.[1]
- **Specification:** docs/ops/specs/binaries/help.md (Exists).
- **Certification Status: Functional & Specified.** Supports "Absolute Literalism" by

allowing agents to verify definitions.
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

3.4 ops/bin/llms: The Context Bundler

- **Function:** Compiles specific subsets of the repository (scope bundles) for LLM consumption.[1]
- **Specification:** docs/ops/specs/binaries/llms.md (Exists).
- **Certification Status: Certified.** Implements critical anti-hallucination guardrails (Hazard Exclusion).
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

3.5 ops/bin/map: The Wayfinding Maintainer

- **Function:** Regenerates the directory index block in docs/MAP.md.[1]
- **Specification:** docs/ops/specs/binaries/map.md (Exists).
- **Certification Status: Functional & Specified.**
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

3.6 ops/bin/open: The Timekeeper

- **Function:** Generates the session prompt, establishing the "Freshness Gate".[1]
- **Specification:** docs/ops/specs/binaries/open.md (Exists).
- **Certification Status: Critical/Specified.** As per DP-OPS-0054, correctly regressed to read-only regarding tracked files.
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

3.7 ops/bin/project: The Workflow Orchestrator

- **Function:** Manages the lifecycle of projects/ payloads.[1]
- **Specification:** docs/ops/specs/binaries/project.md (Exists).
- **Certification Status: Functional & Specified.**
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

3.8 ops/bin/prune: The Hygiene Engine

- **Function:** Manages the archiving of SoP.md entries and cleans storage/.[1]
- **Specification:** docs/ops/specs/binaries/prune.md (Exists).
- **Certification Status: Certified.** Enforces "Equilibrium Maintenance."
- **Registry Status: Missing.** Requires entry in docs/ops/registry/BINARIES.md.

4. Verification and Immune System Certification (tools/)

The tools/ directory contains the "Immune System." These scripts enforce internal constraints. Unlike the Operator Interface (ops/bin), these tools **do not require narrative specifications**. Their "truth" is defined by the code itself. However, to satisfy the "No Ghost" governance requirement, they **MUST be registered**.

4.1 Linting Suite (tools/lint/)

- **Files:** agent.sh, context.sh, dp.sh, library.sh, llms.sh, project.sh, style.sh, task.sh, truth.sh.[1]
- **Function:** Immunological defense against drift, style violations, and context hazards.
- **Specification Status: N/A.** Internal enforcement tools do not require specs.
- **Registry Status: Missing.** Requires entry in docs/ops/registry/LINT.md to prevent ghost files in the immune system.

4.2 Testing and Verification

- **Files:** tools/test/agent.sh, tools/verify.sh.[1]
- **Function:** Deep integrity checks and repo hygiene verification.
- **Specification Status: N/A.**
- **Registry Status: Missing.** Requires entries in docs/ops/registry/TEST.md and docs/ops/registry/TOOLS.md.

5. Logic Subsystem and Script Analysis (ops/lib/scripts)

These scripts (agent.sh, heuristics.sh, project.sh, skill.sh, task.sh) provide reusable logic libraries.

- **Function:** Shared logic for binaries and linters.
- **Specification Status: N/A.** These are internal libraries, not operator-facing commands.
- **Registry Status: Missing.** Requires entry in docs/ops/registry/SCRIPTS.md.

6. The Library: Agents, Tasks, and Skills

The "Explain" layer is populated by registered agents, tasks, and skills. These subsystems are currently the only ones achieving full certification status, as they are fully functional, specified (via their own definitions), and registered.

6.1 Agents (The Workforce)

- **Registry:** docs/ops/registry/AGENTS.md (Exists and populated).
- **Certification: PASS.** All agents (R-AGENT-01 to R-AGENT-06) are registered and aligned with the filesystem.[1]

6.2 Tasks (The Routines)

- **Registry:** docs/ops/registry/TASKS.md (Exists and populated).
- **Certification: PASS.** All tasks (B-TASK-01 to B-TASK-06) are registered and adhere to the pointer-first schema.[1]

6.3 Skills (The Knowledge)

- **Registry:** docs/ops/registry/SKILLS.md (Exists and populated).
- **Certification: PASS.** All skills (S-LEARN-01 to S-LEARN-06) are registered.[1]

7. Registry Gap Analysis and Compliance Findings

The review confirms that the repository adheres to "Pointer-First" logic for the library content but fails to apply this same rigor to the toolchain itself. This creates a "Ghost Governance" risk where executable code exists without being accounted for in a central ledger.

Corrected Doctrine: Specification vs. Registration

- **Operator Interface (ops/bin):** MUST be Specified (Done) AND Registered (Pending).
- **Internal Enforcement (tools/ & ops/lib):** MUST be Registered (Pending). Narrative specs are NOT required.

Missing Registries

The following files must be created to achieve full certification:

1. docs/ops/registry/BINARIES.md
2. docs/ops/registry/LINT.md
3. docs/ops/registry/TOOLS.md
4. docs/ops/registry/TEST.md
5. docs/ops/registry/SCRIPTS.md

8. Certification Matrix

The following table summarizes the revised certification status.

Component Category	Path	Functional?	Specified?	Registered?	Verdict
Binaries	ops/bin/*	YES	YES	**NO**	**Pending Registration**
Linters	tools/lint/*	YES	N/A	**NO**	**Pending Registratio**

					n
Scripts	ops/lib/scripts/*	YES	N/A	**NO**	**Pending Registration**
Root Tools	tools/*	YES	N/A	**NO**	**Pending Registration**
Agents	docs/library/agents/*	YES	YES	YES	**CERTIFIED**
Tasks	docs/library/tasks/*	YES	YES	YES	**CERTIFIED**
Skills	docs/library/skills/*	YES	YES	YES	**CERTIFIED**

9. Recommendations for Full Certification

To achieve the "Explainable" standard required by the user, the following actions must be taken. No new code or specifications are needed; strictly bureaucratic registration.

Action 1: Instantiate Missing Registries

Create the following five files in docs/ops/registry/. Each file must map the file path to its purpose, creating a definitive "Allowlist" for executable code.

1. docs/ops/registry/BINARIES.md

- **Scope:** All executable binaries in ops/bin/.
- **Schema:** | Binary Name | Path | Spec Pointer | Description |
- **Requirement:** Must link to existing specs in docs/ops/specs/binaries/.

2. docs/ops/registry/LINT.md

- **Scope:** All scripts in tools/lint/.
- **Schema:** | Linter Name | Path | Enforcement Domain |
- **Requirement:** Brief description of what it polices (e.g., "Enforces PoT spelling").

3. docs/ops/registry/TOOLS.md

- **Scope:** Root tools like tools/verify.sh.
- **Schema:** | Tool Name | Path | Purpose |

4. docs/ops/registry/TEST.md

- **Scope:** All scripts in tools/test/.
- **Schema:** | Test Name | Path | Target Component |

5. docs/ops/registry/SCRIPTS.md

- **Scope:** Shared logic libraries in ops/lib/scripts/.
- **Schema:** | Script Name | Path | Logic Domain |

Action 2: Enforce "No Ghost" Policy

Once these registries are created, the tools/lint/library.sh script (the Library Guard) should eventually be updated to scan these new registries. This will ensure that if a new script is added to tools/lint/ without being added to LINT.md, the build will fail. This creates a "closed loop" governance system where code cannot exist without a bureaucratic record.

Final Verdict: The Stela repository is **Functionally Certified** but **Bureaucratically Incomplete**. By creating the five missing registries outlined above, the system will achieve total "Pointer-First" compliance, rendering every line of executable code fully explainable and accounted for.

Works cited

1. dump-full-main-d760e443.txt